Paradigmas de Programación [1]

Cálculo-λ [2]

2do cuatrimestre de 2025 [3]

Departamento de Computación [4]
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Introducción 1

Cálculo-λ^b: sintaxis y tipado 2

Cálculo-λ^b: semántica operacional 3

Cálculo-λ^{bn}: extensión con números naturales 5

¿Qué es el cálculo-λ? 1

Lenguaje de programación definido de manera rigurosa. 2
Se basa sólo en dos operaciones: construir funciones y aplicarlas.

Históricamente 3

- Concebido en la década de 1930 por Alonzo Church para formalizar la noción de función efectivamente computable. 4
- Usado desde la década de 1960 para estudiar semántica formal de lenguajes de programación.

Actualmente 5

- Núcleo de lenguajes de programación funcionales y asistentes de demostración. 6
  Lisp, OCaml, Haskell, Coq, Agda, Lean, ...
- Laboratorio para investigar nuevas características de lenguajes. 7
- Fuertemente conectado con la teoría de la demostración, matemática constructiva, teoría de categorías, ... 8

Introducción 1

Cálculo-λ^b: sintaxis y tipado 2

Cálculo-λ^b: semántica operacional 3

Cálculo-λ^{bn}: extensión con números naturales 5

El cálculo-λ^b 1

Sintaxis de los tipos 2

$$\begin{array}{rcl} \tau, \sigma, \rho, \ldots & ::= & \mathsf{bool} \\ & | & \tau \to \sigma \end{array}$$ 3

Asumimos que el constructor de tipos "$\to$" es asociativo a derecha: 4

$$\tau \to \sigma \to \rho \quad = \quad \tau \to (\sigma \to \rho) \quad \neq \quad (\tau \to \sigma) \to \rho$$ 5

El cálculo-λ^b 1

Suponemos dado un conjunto infinito numerable de variables: 2

$$\mathcal{X} = \{x, y, z, \ldots\}$$ 3

Sintaxis de los términos 4

$M, N, P, \ldots$	$::=$	x	variable	5
	$\mid$	$\lambda x : \tau.\, M$	abstracción	
	$\mid$	$M\, N$	aplicación	
	$\mid$	true	verdadero	
	$\mid$	false	falso	
	$\mid$	if M then N else P	condicional	

Asumimos que la aplicación es asociativa a izquierda: 6

$$M\, N\, P \quad = \quad (M\, N)\, P \quad \neq \quad M\, (N\, P)$$ 7

La abstracción y el "if" tienen menor precedencia que la aplicación: 8

$$\lambda x : \tau.\, M\, N \quad = \quad \lambda x : \tau.\, (M\, N) \quad \neq \quad (\lambda x : \tau.\, M)\, N$$ 9

El cálculo-λ^b

1

Ejemplos de términos

- $\lambda x : \mathsf{bool}.\, x$
- $\lambda x : \mathsf{bool} \to \mathsf{bool}.\, x$
- $(\lambda x : \mathsf{bool}.\, x)\, \mathsf{false}$
- $(\lambda x : \mathsf{bool} \to \mathsf{bool}.\, x)\, (\lambda y : \mathsf{bool}.\, y)$
- $(\lambda x : \mathsf{bool}.\, \lambda y : \mathsf{bool} \to \mathsf{bool}.\, y\, x)\, \mathsf{true}$
- $\lambda x : \mathsf{bool}.\, \mathsf{if}\ x\ \mathsf{then}\, \mathsf{false}\ \mathsf{else}\ \mathsf{true}$
- $\mathsf{true}\, \mathsf{true}$
- $\mathsf{if}\ \lambda x : \mathsf{bool}.\, x\ \mathsf{then}\, \mathsf{false}\ \mathsf{else}\ \mathsf{true}$

Variables libres y ligadas 1

Una ocurrencia de x está **ligada** si aparece adentro de una abstracción "$\lambda\, x$". Una ocurrencia de x está **libre** si no está ligada. 2

Ejemplo 3

Marcar ocurrencias de variables libres y ligadas: 4

$$(\lambda x : \mathsf{bool} \to \mathsf{bool}.\, \lambda y : \mathsf{bool}.\, x\, y)\, (\lambda y : \mathsf{bool}.\, x\, y)\, y$$ 5

Ejercicio 6

Definir el conjunto de variables libres $\mathsf{fv}(M)$ de M. 7

Alfa equivalencia 8

Los términos que difieren sólo en el nombre de variables *ligadas* se consideran iguales: 9

$$\begin{array}{lcl} \lambda x : \tau.\, \lambda y : \sigma.\, x & = & \lambda y : \tau.\, \lambda x : \sigma.\, y \;=\; \lambda a : \tau.\, \lambda b : \sigma.\, a \\ \lambda x : \tau.\, \lambda y : \sigma.\, x & \neq & \lambda x : \tau.\, \lambda y : \sigma.\, y \;=\; \lambda x : \tau.\, \lambda x : \sigma.\, x \end{array}$$ 10

Sistema de tipos 1

La noción de "tipabilidad" se formaliza con un sistema deductivo. 2

Problema 3

¿Qué tipo tiene x? 4

Contextos de tipado 5

Un **contexto de tipado** es un conjunto finito de pares $(x_i : \tau_i)$: 6

$$\{x_1 : \tau_1, \ldots, x_n : \tau_n\} \quad 7$$

sin variables repetidas $(i \neq j \Rightarrow x_i \neq x_j)$. 8
Se nota con letras griegas mayúsculas $(\Gamma, \Delta, \ldots)$. 9
A veces notamos $\mathrm{dom}(\Gamma) = \{x_1, \ldots, x_n\}$. 10

Juicios de tipado 11

El sistema de tipos predica sobre **juicios de tipado**, de la forma: 12

$$\Gamma \vdash M : \tau$$

Sistema de tipos 1

Reglas de tipado 2

$$\frac{}{\Gamma \vdash \mathsf{true} : \mathsf{bool}}\,\text{T-TRUE} \qquad \frac{}{\Gamma \vdash \mathsf{false} : \mathsf{bool}}\,\text{T-FALSE}$$ 3

$$\frac{\Gamma \vdash M : \mathsf{bool} \quad \Gamma \vdash N : \tau \quad \Gamma \vdash P : \tau}{\Gamma \vdash \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P : \tau}\,\text{T-IF}$$

$$\frac{}{\Gamma, x : \tau \vdash x : \tau}\,\text{T-VAR} \qquad \frac{\Gamma, x : \tau \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau.\, M : \tau \to \sigma}\,\text{T-ABS}$$

$$\frac{\Gamma \vdash M : \tau \to \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M\, N : \sigma}\,\text{T-APP}$$

Sistema de tipos 1

Ejemplo — derivaciones de juicios de tipado 2

Derivar, si es posible, juicios de tipado para los siguientes términos: 3

1. $\lambda x : \mathsf{bool}.\, \mathsf{if}\ x\ \mathsf{then}\, \mathsf{false}\ \mathsf{else}\ x$ 4
2. $\lambda y : \mathsf{bool} \to \mathsf{bool} \to \mathsf{bool}.\, \lambda z : \mathsf{bool}.\, y\,(y\,x\,z)$
3. $x\,z\,(y\,z)$
4. $\lambda x : \mathsf{bool} \to \mathsf{bool}.\, \lambda x : \mathsf{bool}.\, x$
5. $\mathsf{true}\,(\lambda x : \mathsf{bool}.\, x)$
6. $x\,x$

Propiedades del sistema de tipos [1]

Teorema (Unicidad de tipos) [2]

Si $\Gamma \vdash M : \tau$ y $\Gamma \vdash M : \sigma$ son derivables, entonces $\tau = \sigma$. [3]

Teorema (Weakening + Strengthening) [4]

Si $\Gamma \vdash M : \tau$ es derivable y $\mathrm{fv}(M) \subseteq \mathrm{dom}(\Gamma \cap \Gamma')$ entonces [5] $\Gamma' \vdash M : \tau$ es derivable.

Introducción 1

Cálculo-λ^b: sintaxis y tipado 2

Cálculo-λ^b: semántica operacional 3

Cálculo-λ^{bn}: extensión con números naturales 4

Semántica formal 1

El sistema de tipos indica cómo se construyen los programas. 2
Queremos además darles **significado** (semántica).

Distintas maneras de dar semántica formal 3

1. **Semántica operacional.** 4
   Indica cómo se ejecuta el programa hasta llegar a un resultado. 5

   Semántica *small-step*: ejecución paso a paso. 6
   Semántica *big-step*: evaluación directa al resultado.

2. **Semántica denotacional.** 7
   Interpreta los programas como objetos matemáticos.

3. **Semántica axiomática.** 8
   Establece relaciones lógicas entre el estado del programa antes y después de la ejecución.

4. ... 9

Vamos a trabajar con semántica operacional *small-step*. 10

Semántica operacional *small-step* 1

Programas 2

Un **programa** es un término M tipable y *cerrado* ($\mathrm{fv}(M) = \emptyset$): 3

- El juicio de tipado $\vdash M : \tau$ debe ser derivable para algún τ. 4

Juicios de evaluación 5

La semántica operacional predica sobre **juicios de evaluación**: 6

$$M \to N$$

donde M y N son programas. 7

Valores 8

Los **valores** son los posibles resultados de evaluar programas: 9

$$V ::= \mathsf{true} \mid \mathsf{false} \mid \lambda x : \tau.\, M$$

Semántica operacional *small-step* 1

Reglas de evaluación para expresiones booleanas 2 3

$$\frac{}{\text{if true then } M \text{ else } N \to M}\ \textsc{e-ifTrue}$$

$$\frac{}{\text{if false then } M \text{ else } N \to N}\ \textsc{e-ifFalse}$$

$$\frac{M \to M'}{\text{if } M \text{ then } N \text{ else } P \to \text{if } M' \text{ then } N \text{ else } P}\ \textsc{e-if}$$

Semántica operacional *small-step* 1

Ejemplo 2

1. Derivar el siguiente juicio: 3

$$\begin{array}{ll} & \text{if (if false then false else true) then false else true} \\ \rightarrow & \text{if true then false else true} \end{array}$$ 4

2. ¿Para qué términos M vale que $\text{true} \rightarrow M$? 5
3. ¿Es posible derivar el siguiente juicio? 6

$$\begin{array}{ll} & \text{if true then (if false then false else false) else true} \\ \rightarrow & \text{if true then false else true} \end{array}$$ 7

Semántica operacional *small-step* 1

Reglas de evaluación para funciones (abstracción y aplicación) 2

3

$$\frac{M \to M'}{M\,N \to M'\,N}\,\textsc{e-app1}$$

$$\frac{N \to N'}{(\lambda x : \tau.\, M)\,N \to (\lambda x : \tau.\, M)\,N'}\,\textsc{e-app2}$$

$$\frac{}{(\lambda x : \tau.\, M)\,V \to M\{x := V\}}\,\textsc{e-appAbs}$$

Sustitución 1

La operación de **sustitución**: 2

$$M\{x := N\}$$ 3

denota el término que resulta de reemplazar todas las ocurrencias 4 libres de x en M por N.

Sustitución

Definición de sustitución 1

2

$$
\begin{aligned}
x\{x := N\} &\stackrel{\text{def}}{=} N \\
a\{x := N\} &\stackrel{\text{def}}{=} a \ \text{ si } a \in \{\mathsf{true}, \mathsf{false}\} \cup \mathcal{X} \setminus \{x\} \\
(\text{if } M \text{ then } P \text{ else } Q)\{x := N\} &\stackrel{\text{def}}{=} \text{if } M\{x := N\} \\
&\quad \text{then } P\{x := N\} \\
&\quad \text{else } Q\{x := N\} \\
(M_1\, M_2)\{x := N\} &\stackrel{\text{def}}{=} M_1\{x := N\}\, M_2\{x := N\} \\
(\lambda y : \tau.\, M)\{x := N\} &\stackrel{\text{def}}{=} \\
&\begin{cases}
\lambda y : \tau.\, M & \text{si } x = y \\
\lambda y : \tau.\, M\{x := N\} & \text{si } x \neq y,\ y \notin \mathsf{fv}(N) \\
\lambda z : \tau.\, M\{y := z\}\{x := N\} & \text{si } x \neq y,\ y \in \mathsf{fv}(N), \\
& \quad z \notin \{x, y\} \cup \mathsf{fv}(M) \cup \mathsf{fv}(N)
\end{cases}
\end{aligned}
$$

Sustitución

Definición de sustitución (alternativa) 1

2

$$
\begin{array}{rcl}
x\{x := N\} & \stackrel{\text{def}}{=} & N \\
a\{x := N\} & \stackrel{\text{def}}{=} & a \ \text{ si } a \in \{\text{true}, \text{false}\} \cup \mathcal{X} \setminus \{x\} \\
(\text{if } M \text{ then } P \text{ else } Q)\{x := N\} & \stackrel{\text{def}}{=} & \text{if } M\{x := N\} \\
 & & \ \text{then } P\{x := N\} \\
 & & \ \text{else } Q\{x := N\} \\
(M_1\, M_2)\{x := N\} & \stackrel{\text{def}}{=} & M_1\{x := N\}\, M_2\{x := N\} \\
(\lambda y : \tau.\, M)\{x := N\} & \stackrel{\text{def}}{=} & \lambda y : \tau.\, M\{x := N\} \\
 & & \ \text{asumiendo } y \notin \{x\} \cup \text{fv}(N)
\end{array}
$$

La asunción se puede cumplir siempre, renombrando la variable ligada "y" en caso de conflicto. 3

Semántica operacional *small-step* 1

Definimos un **juicio de evaluación en muchos pasos**: 2

$$M \twoheadrightarrow N$$

donde M y N son programas. 3

Reglas de evaluación en muchos pasos 4

$$\frac{}{M \twoheadrightarrow M} \qquad \frac{M \to N \qquad N \twoheadrightarrow P}{M \twoheadrightarrow N}$$ 5

Semántica operacional *small-step* 1

Ejemplo — evaluación 2

Reducir repetidamente el siguiente término hasta llegar a un valor: 3

$$(\lambda x : \text{bool}.\, \lambda f : \text{bool} \to \text{bool}.\, f\,(f\,x))\,\text{true}\,(\lambda x : \text{bool}.\, x)$$ 4

Propiedades de la evaluación 1

Teorema (Determinismo) 2

Si $M \to N_1$ y $M \to N_2$ entonces $N_1 = N_2$. 3

Teorema (Preservación de tipos) 4

Si $\vdash M : \tau$ y $M \to N$ entonces $\vdash N : \tau$. 5

Teorema (Progreso) 6

Si $\vdash M : \tau$ entonces: 7

1. O bien M es un valor. 8
2. O bien existe N tal que $M \to N$.

Teorema (Terminación) 9

Si $\vdash M : \tau$, entonces no hay una cadena infinita de pasos: 10

$$M \to M_1 \to M_2 \to \ldots$$ 11

Propiedades de la evaluación 1

Corolario (Canonicidad) 2

1. Si $\vdash M : \text{bool}$ es derivable, entonces la evaluación de M termina y el resultado es true o false. 3
2. Si $\vdash M : \tau \to \sigma$ es derivable, entonces la evaluación de M termina y el resultado es una abstracción. 4

Slogan 5

Well typed programs cannot go wrong. 6 (Robin Milner) 7

Introducción 1

Cálculo-λ^b: sintaxis y tipado 2

Cálculo-λ^b: semántica operacional 3

Cálculo-λ^{bn}: extensión con números naturales 4

El cálculo λ^{bn} 1

Sintaxis: tipos 2

$$\tau, \sigma, \ldots \quad ::= \quad \ldots \mid \text{nat}$$ 3

Sintaxis: términos 4

Semántica informal 5

M	$::=$	$\ldots$	6
	$\mid$	zero	el número cero
	$\mid$	succ(M)	el sucesor del número que representa M
	$\mid$	pred(M)	el predecesor del número que representa M
	$\mid$	isZero(M)	representa un booleano true/false, dependiendo de si M representa al cero o no

7

El cálculo λ^{bn}: reglas de tipado [1]

$$\frac{}{\Gamma \vdash \mathsf{zero} : \mathsf{nat}}\ \textsc{T-ZERO}$$ [2]

$$\frac{\Gamma \vdash M : \mathsf{nat}}{\Gamma \vdash \mathsf{succ}(M) : \mathsf{nat}}\ \textsc{T-SUCC} \qquad \frac{\Gamma \vdash M : \mathsf{nat}}{\Gamma \vdash \mathsf{pred}(M) : \mathsf{nat}}\ \textsc{T-PRED}$$ [3]

$$\frac{\Gamma \vdash M : \mathsf{nat}}{\Gamma \vdash \mathsf{isZero}(M) : \mathsf{bool}}\ \textsc{T-isZero}$$ [4]

El cálculo λ^{bn}: valores [1]

Extendemos el conjunto de **valores**: [2]

$$V ::= \ldots \mid \mathsf{zero} \mid \mathsf{succ}(V)$$ [3]

El cálculo λ^{bn}: semántica operacional 1

2

$$\frac{M \to M'}{\mathsf{succ}(M) \to \mathsf{succ}(M')}\ \textsc{e-succ}$$

$$\frac{M \to M'}{\mathsf{pred}(M) \to \mathsf{pred}(M')}\ \textsc{e-pred}$$

$$\frac{}{\mathsf{pred}(\mathsf{succ}(V)) \to V}\ \textsc{e-predSucc}$$

$$\frac{M \to M'}{\mathsf{isZero}(M) \to \mathsf{isZero}(M')}\ \textsc{e-isZero}$$

$$\frac{}{\mathsf{isZero}(\mathsf{zero}) \to \mathsf{true}}\ \textsc{e-isZeroZero}$$

$$\frac{}{\mathsf{isZero}(\mathsf{succ}(V)) \to \mathsf{false}}\ \textsc{e-isZeroSucc}$$

El cálculo λ^{bn}: semántica operacional 1

Ejemplo 2

1. Evaluar isZero(succ(pred(succ(zero)))). 3
2. Evaluar isZero(pred(pred(succ(zero)))). (¿Qué ocurre?) 4

Forma normal ("f.n.") 5

Un programa M es una **f.n.** si no existe M' tal que $M \to M'$. 6

¿Todas las f.n.'s cerradas y tipables son valores? 7

En el cálculo-λ^b **sí**.
En el cálculo-λ^{bn} **no**. (¿Qué propiedad deja de valer?) 8

Las f.n.'s que no son valores se llaman **términos de error**. 9

¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ? ? ? ? ? ? ? ? ? 1

Lectura recomendada 2

Capítulos 5 y 9 del libro de Pierce.

Benjamin C. Pierce. *Types and Programming Languages*
The MIT Press, 2002.